

ONE-PIECE

A forensic reconstruction of a steganography challenge

EVIDENCE	meme.png	CATEGORY	Forensics
METHOD	Metadata -> LSB -> carving -> decryption	ANALYST	Het Dalal

MISSION

Recover the genuine flag without falling for the challenge's deliberately flag-shaped decoy. The important part of this POC is not the final string; it is the evidence chain that proves how the string was recovered.

01	Metadata
02	LSB clue
03	Appended ZIP
04	Carve + decrypt
05	Luffy + Zoro

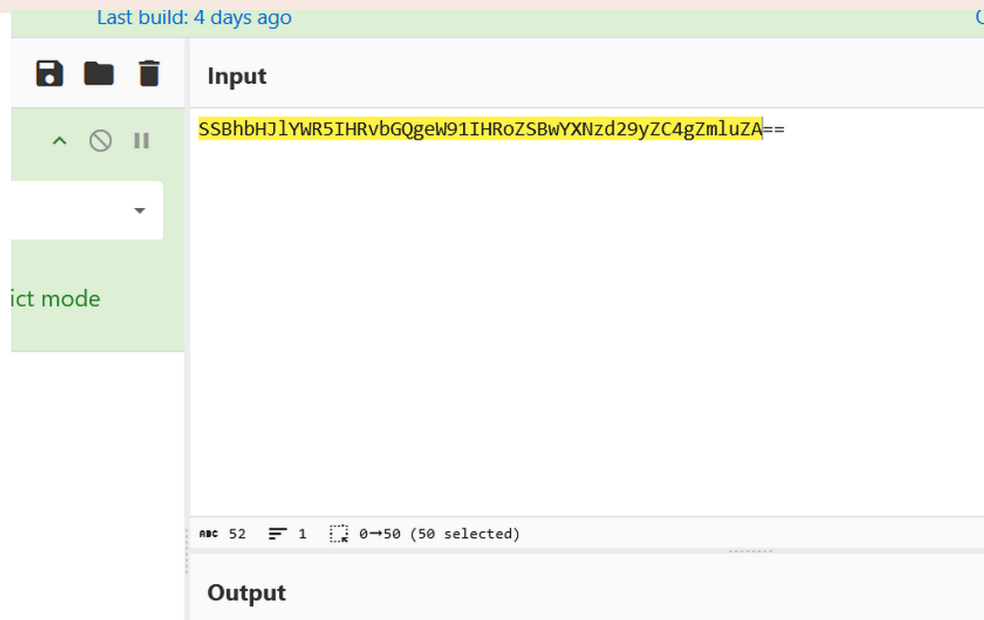
01 / Metadata gives the direction

The first clue is not hidden deep in the file. Start with metadata.

```
exiftool meme.png
```

The PNG Comment contains Base64. Decoding it returns:

I already told you the password. find



Evidence A - Base64 comment decoded. The screenshot is cropped to the relevant input/output area for readability.

Interpretation

This is a direction, not a password. It tells us that a password has already been exposed somewhere in the evidence path.

The carrier image is checked for least-significant-bit steganography.

```
zsteg meme.png zsteg -a meme.png
```

The LSB content produces a flag-shaped decoy containing the value 12k974a0. The challenge description warns that finding something that looks like treasure does not mean it is the correct flag. Combined with the metadata clue, the value is treated as a password candidate.

Password candidate: 12k974a0

Why not submit the first H4S-CTF-looking string? Because the narrative itself tells us that this is the trap. A forensic solve should correlate clues instead of accepting a familiar pattern at face value.

Structural validation exposes the hidden container

```
pngcheck -v meme.png
```

```
no vm guests are running outdated hypervisor (qemu) binaries on this host.
(root@kali)-[/home/kali/Desktop]
# pngcheck -v meme.png
zlib warning: different version (expected 1.3.1, using 1.3.2)

File: meme.png (1688123 bytes)
chunk IHDR at offset 0x0000c, length 13
  442 x 438 image, 24-bit RGB, non-interlaced
chunk tEXt at offset 0x00025, length 60, keyword: Comment
chunk IDAT at offset 0x0006d, length 65536
  zlib: deflated, 32K window, default compression
chunk IDAT at offset 0x10079, length 65536
chunk IDAT at offset 0x20085, length 65536
chunk IDAT at offset 0x30091, length 39880
chunk IEND at offset 0x39c65, length 0
additional data after IEND chunk
ERRORS DETECTED in meme.png

(root@kali)-[/home/kali/Desktop]
# strings -a meme.png | grep -iE 'pass|password|luffy|zoro|find|key|one|piece'
luffy.pngUT
zoro.pngUT
KEYUS{=
luffy.pngUT
zoro.pngUT
```

Evidence B - pngcheck reports additional data after IEND. That is the structural clue that another payload was appended.

A normal PNG terminates at the IEND chunk. Extra bytes after IEND are therefore investigated as appended data rather than treated as ordinary image content.

```
binwalk meme.png
```

```
hidden.zip.zip, and cannot find hidden.zip.zip, period.
(root@kali)-[/home/kali/Desktop]
# binwalk meme.png

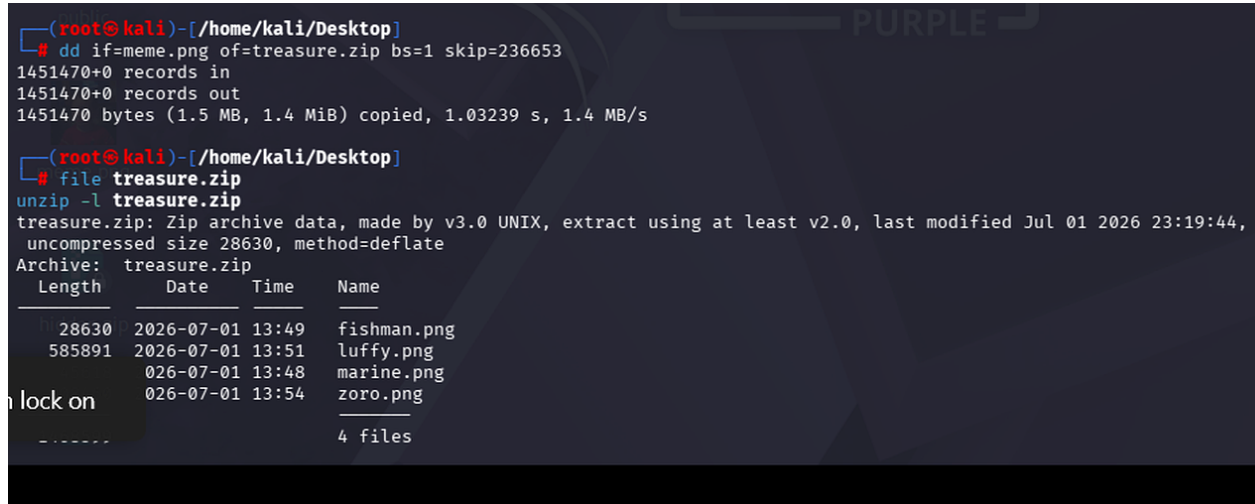
DECIMAL      HEXADECIMAL  DESCRIPTION
-----
0            0x0         PNG image, 442 x 438, 8-bit/color RGB, non-interlaced
113          0x71        Zlib compressed data, default compression
236653       0x39C6D     Zip archive data, encrypted at least v2.0 to extract, compressed size: 28609, uncompressed size: 28630, name: fishman.png
265347       0x40C83     Zip archive data, encrypted at least v2.0 to extract, compressed size: 568779, uncompressed size: 585891, name: luffy.png
834209       0xCBAA1     Zip archive data, encrypted at least v2.0 to extract, compressed size: 45426, uncompressed size: 45618, name: marine.png
879719       0xD6C67     Zip archive data, encrypted at least v2.0 to extract, compressed size: 807982, uncompressed size: 808460, name: zoro.png
1688101      0x19C225    End of Zip archive, footer length: 22
```

Evidence C - Binwalk locates an encrypted ZIP at decimal offset 236653 / 0x39C6D and identifies four PNG entries.

Extract only the bytes that belong to the appended ZIP

```
dd if=meme.png of=treasure.zip bs=1 skip=236653 file treasure.zip unzip -l treasure.zip
```

dd skips the carrier portion and writes the bytes beginning at the ZIP offset to a new file. file is then used as a validation step; this prevents us from forcing a bad extraction just because a tool reported a ZIP signature.



```
(root@kali)-[/home/kali/Desktop]
# dd if=meme.png of=treasure.zip bs=1 skip=236653
1451470+0 records in
1451470+0 records out
1451470 bytes (1.5 MB, 1.4 MiB) copied, 1.03239 s, 1.4 MB/s

(root@kali)-[/home/kali/Desktop]
# file treasure.zip
unzip -l treasure.zip
treasure.zip: Zip archive data, made by v3.0 UNIX, extract using at least v2.0, last modified Jul 01 2026 23:19:44,
uncompressed size 28630, method=deflate
Archive:  treasure.zip
  Length      Date    Time    Name
  ----      -
  28630   2026-07-01 13:49   fishman.png
  585891  2026-07-01 13:51   luffy.png
         2026-07-01 13:48   marine.png
  1 lock on 2026-07-01 13:54   zoro.png
  -----
                   4 files
```

Evidence D - treasure.zip is successfully carved and its four entries are listed.

Decrpyt

```
unzip -P '12k974a0' treasure.zip
```

The password works, confirming the LSB interpretation. The archive yields fishman.png, luffy.png, marine.png and zoro.png.

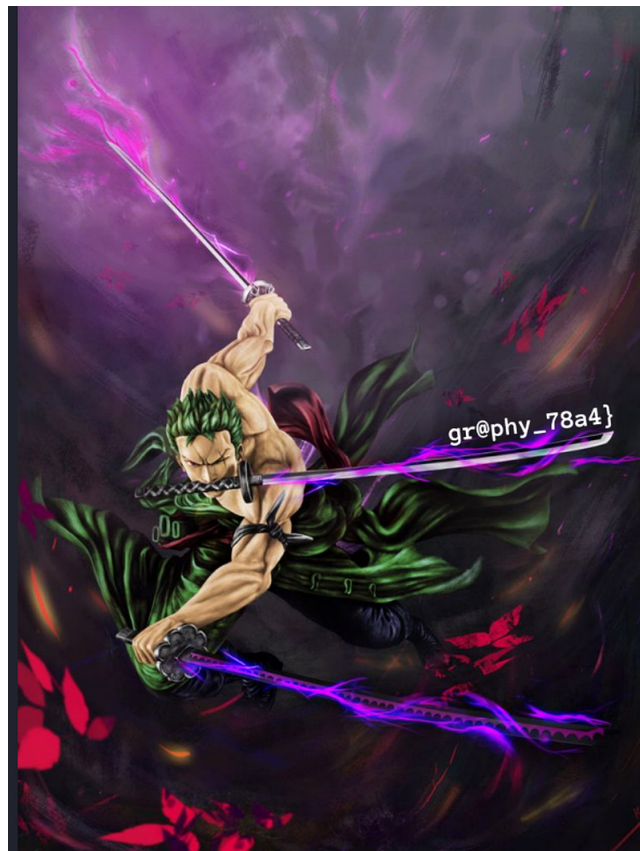
05 / The final hint is literal

Luffy + Zoro = the real flag

The challenge's third hint tells us which two recovered images matter. The first fragment is printed vertically on the far-left edge of Luffy; the second is printed over the Zoro artwork.



Evidence E - Luffy: H4S-CTF {st3gn0}



Evidence F - Zoro: gr@phy_78a4}

RECONSTRUCTION

The two fragments are concatenated - no decoding step remains.

Luffy : H4S-CTF{st3gn0 Zoro : gr@phy_78a4} FINAL : H4S-CTF{st3gn0gr@phy_78a4}

H4S-CTF{st3gn0gr@phy_78a4}

WHAT THIS CHALLENGE TESTED

Metadata: recognize and decode an embedded clue.

Steganography: use LSB analysis and distinguish a decoy from a credential.

File structure: recognize data after PNG IEND.

Carving: recover an appended ZIP from a known byte offset.

Correlation: use the narrative hint to combine evidence from two separate images.

Analyst note

The early false path was trying to turn every zsteg ZIP signature into an archive. The correct habit is simple: validate every extraction. If `file` says only `data`, the hypothesis is not yet proven. Structural evidence from pngcheck and binwalk provided the defensible route.